

Lab 12 - huggingface

Abstract

Obiective:

- Privire de ansamblu asupra BERT (Bidirectional Encoder Representations from Transformers) model
- Utilizare BERT pentru clasificare
 - sentence encoding + LogisticRegression
 - fine-tuning

1 Language models - BERT

- Fisierul 1_HuggingFace.pdf contine in sectiunile 3,4,5 informatii despre Transfer Learning, Language Model si arhitectura Transformer. Fisierul 2_HuggingFace-code.pdf contine linkuri utile pentru a va reaminti cum functioneaza retele neuronale si pentru a afla care e ideea din spatele trecerii de la reprezentare discreta (nivelul cuvintelor) la reprezentare continua (vectori numerici). O varianta mai veche de reprezentare cuvinte prin numere e word2vec. In word2vec fiecare cuvant e mapat la un vector de numere indiferent de contextul in care apare. Modelele mai noi de limba (language models) adreseaza aceasta problema si permit construirea unor reprezentari ale cuvintelor dependente de context. HuggingFace include o serie de modele de limba preantrenate, inclusiv BERT. Modelele de limba intr-o interpretare simplificata sunt retele neuronale antrenate pentru diverse taskuri (masked language, next sentence) pe seturi foarte mari de date. Ponderile retelelor pot fi ulterior folosite (transfer learning) pentru taskuri particulare in care se porneste de la un set de date mai mic.

- Cum e antrenat BERT?

Pretrainingul s-a facut pe doua taskuri: Masked Language Model (figure 1) si Next sentence task (figure 2) (See Jalammar Illustrated Bert for further details or read [BERT paper](#)).

Bert paper (2018): *"Training of BERTBASE was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total). Training of BERT-LARGE was performed on 16 Cloud TPUs (64 TPU chips total). Each pretraining took 4 days to complete."*

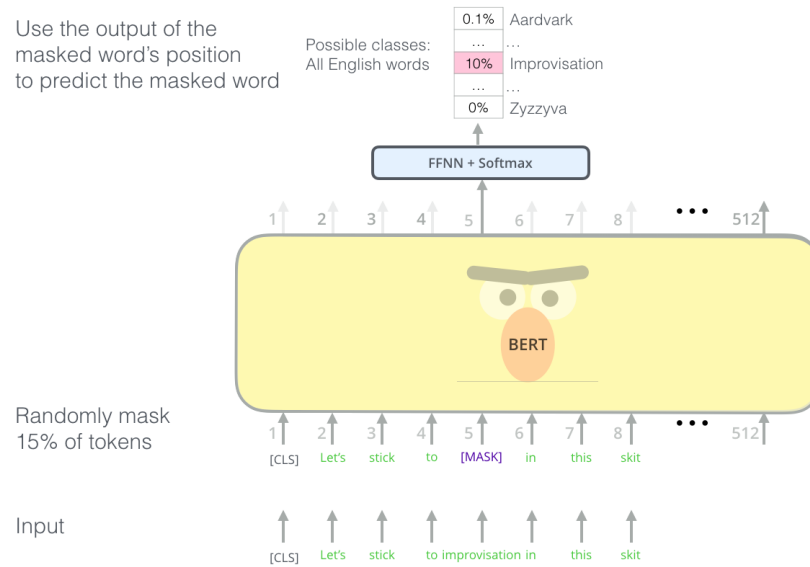


Figure 1: BERT language model task - 15% dintre cuvinte sunt mascate iar modelul trebuie sa le prezica

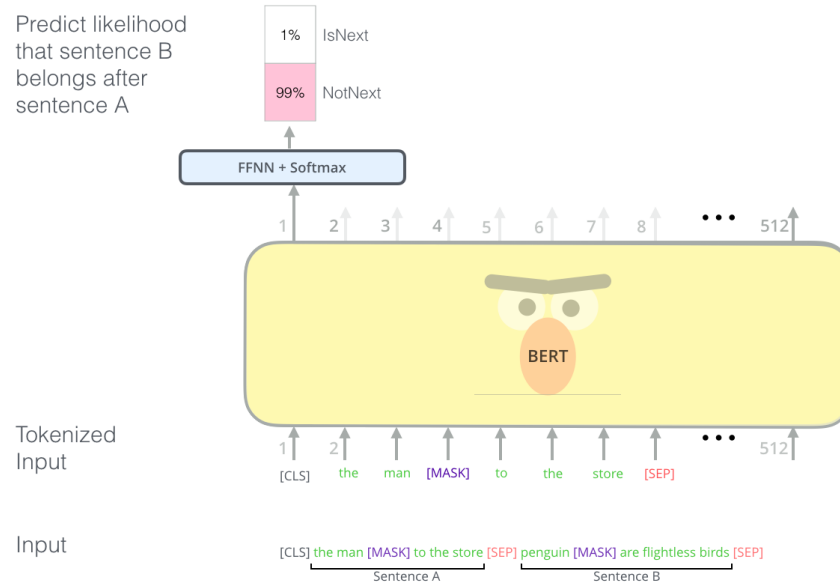


Figure 2: BERT next sentence: Given two sentences (A and B), is B likely to be the sentence that follows A, or not?

- Cum se poate folosi un Language Model?
 - drept embedding - al propozitiei sau al fiecarui token (vezi primul exercitiu)
 - fine tuning pentru diferite taskuri - **varianta recomandata** (figure 3).
(vezi al doilea exercitiu) + HuggingFace Notebooks (How to fine-tune...)

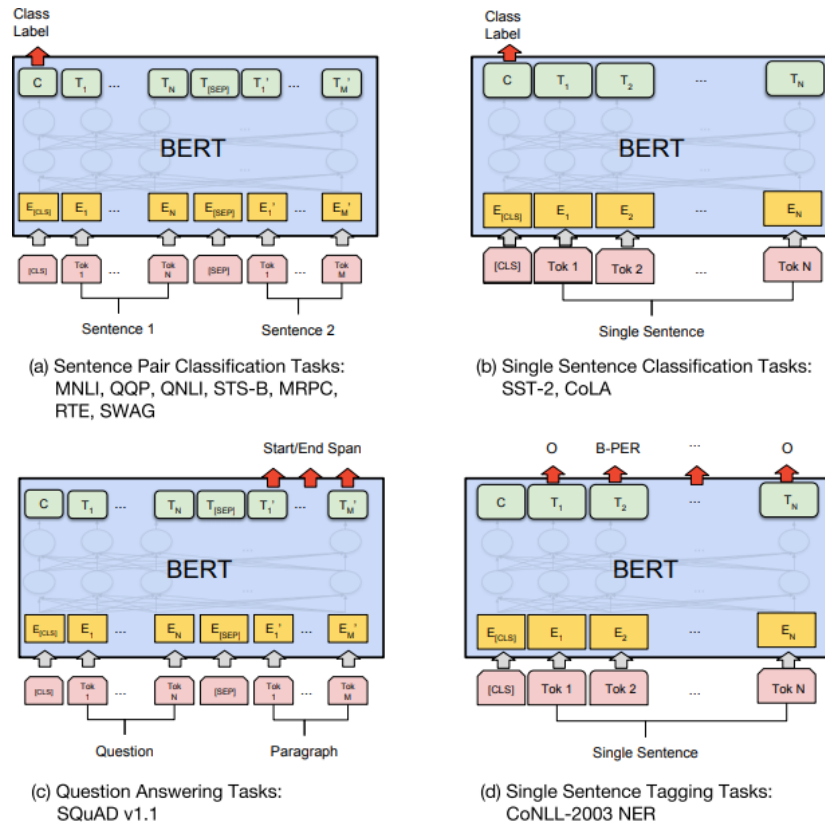


Figure 3: Exemple de utilizare BERT in diverse tipuri de taskuri - atentie la ce anume din outputul modelului se foloseste, respectiv cum se furnizeaza inputul

2 Exerciții

1. Folositi BERT pentru a obtine sentence embedding dupa care LogisticRegression pentru a identifica sentimentul din text. [A visual guide to using BERT](#). Rulati notebookul si adaugati:

- dupa incarcarea modelului, printati arhitectura modelului (prin afisarea modelului) si analizati dimensiunea ultimului layer
 - dupa tokenizare, printati rezultatul tokenizarii primei propozitii: `tokenized[0]`, respectiv `tokenizer.decode(tokenized[0])`. Testati si pentru alta propozitie pentru a intelege rezultatul tokenizarii.
 - dupa aplicarea modelului pe toate propozitiile din setul de date pas-trat, afisati dimensiunea atributului `last_hidden_state` al modelului
- ```
outputs = model(input_ids, attention_mask=attention_mask)
outputs.last_hidden_state.numpy().shape
```

pentru a intelege outputul modelului. Cititi cu atentie comentariile din notebook. Observatie: `outputs` in notebook e numit `last_hidden_states`.

- pentru fiecare propozitie, codificarea propozitiei e vectorul asociat primului token (si anume CLS). Dupe ce faceti antrenarea regresiei logistice cu features depepdent de encodingul pentru CLS, alegeti alt token si refaceti antrenarea. Ar trebui sa observati o scadere a performantei.
2. Cititi [Fine-tuning a pretrained model](#). Pentru fine-tuning se poate folosi clasa speciala `Trainer` din HuggingFace sau antrenare clasica in PyTorch sau Tensorflow.
- Rulati [IMDB classification](#) with `Trainer`. In caz de probleme de memorie reduceti `batch_size` sau schimbati LM la *distilbert-base-uncased*.
  - Aplicati fine-tuning pe datele de la exercitiul 1.

Pentru mai multe informatii pe scurt, cititi [BERT explained](#)